

AIにコードを直させる前に必要だったこと

Jiraチケットから自動修正へつなげる AI Agent 基盤 Qualix の試
行錯誤

manattan

今日話すこと

- コード生成AIを使っていて感じた、次の課題
- Jiraの品管チケットを起点にした自動化を考えた背景
- Qualixで作ったもの
- 実際にやってみて見えてきた難しさ
- AIエージェントを業務に載せるために必要だったこと

まず前提

Claude Code や GitHub Copilot によって、コードを書く体験はかなり変わった。

- 修正方針を伝えると、かなりの速度でコードを書いてくれる
- テスト追加やリファクタも任せやすくなった
- 開発者は「実装する時間」をかなり圧縮できるようになった

ただ、実際の業務では、コードを書き始める前にまだ人間がやっていることが多い。

まだ人間がやっていること

バグチケットが起きたとき、実装に入る前にやることがある。

- チケットに気づく
- 重要度を判断する
- 仕様や過去経緯を追う
- 再現手順を読む
- QA環境やホストを確認する
- Grafana / Loki でログを見る
- サーバー起因か、クライアント起因かを考える
- 関連repoを探す
- AIに説明できる形に整理する

問題意識

AIに依頼するまでが、まだ人間依存

コード生成AIが速くても、そこに到達するまでの調査・整理・判断に時間がかかる。
つまり、AI活用のボトルネックは「コードを書けるか」ではなくなってきた。

AIに作業を頼むための準備作業を、まだ人間が頑張っている。

もう少し現場っぽく言うと

品管チケットでは、こんなことが起きる。

- チケットは起票されているが、気づくのが遅れる
- チケット本文だけでは、どのサービスの問題か分からない
- QAホストや発生環境を読み解く必要がある
- 関連repoがサーバーなのかクライアントなのかすぐ分からない
- ログやSlackの文脈を見ないと判断できない
- GitHub Issueに転記するだけでも地味に手間がかかる

そこで作りたかったもの

Jiraチケットを起点に、AIが初動調査から修正案まで進む仕組み

Jira 品管チケット

- > Qualix が検知
- > チケット内容を解析
- > 対象サービス / repo / 環境を推定
- > Jira / Grafana / GitHub から文脈を収集
- > 原因仮説と修正方針を作成
- > GitHub Issue を作成
- > 将来的にはPR作成まで

人間は、調査の入口ではなく、レビューや判断に集中したい。

Qualixとは

Jira のチケットを AI エージェントが自動で調査し、GitHub Issue や将来的な修正 PR へつなげるシステム。

現在の主な対象は、品管チケット。

- Jira Webhookで起動
- チケット内容からサービス・repoを推定
- MCP経由でJira / Grafana / GitHubを扱う
- Grafana Lokiから関連ログを取得
- 調査結果を含むGitHub Issueを作成
- Jiraに結果をコメント

目指した価値

Qualixでやりたかったのは、単に「AIにコードを書かせる」ことではない。

- チケットに気づく
- 調査のあたりをつける
- 必要な情報を集める
- GitHub Issueとして開発者が動ける形にする
- 将来的には修正PRまでつなげる

つまり、コードを書く前の初動をAIに寄せたかった。

現状アーキテクチャ

```
Jira Webhook
  |
  v
Echo Server (/webhooks/jira)
  |
  v
Ticket Filter / Classifier
  |
  v
OpenAI Agent Loop
  |
  +-- Jira MCP
  +-- Grafana MCP
  +-- GitHub MCP
  |
  v
GitHub Issue / Jira Comment
```

実装の現在地

コードとしては、PoCから実運用手前まで形になっている。

- Jira Webhook受信
- HMAC-SHA256署名検証
- チケットフィルタ
- 軽量AI分類
- MCP tool収集
- allowed / denied toolsによる制御
- OpenAI Agent loop
- service catalogによるrepo / namespace / datasource制御

一方で、今は「実運用に載せるための調整・検証フェーズ」。

重要な設計: AIに自由生成させすぎない

AIにrepo名やnamespaceを自由に作らせると危険。

そのため、Qualixでは YAML の service catalog を single source of truth として扱う。

```
repositories:  
  - service: qualix  
    repository: manattan/qualix  
    jira_components: ["qualix"]  
    environments:  
      qa:  
        namespace: qualix-qa  
        grafana_datasource: loki-qa
```

AIには「選ばせる」。決定論的に解決できる値はコードと設定で縛る。

難しかったこと1: どこを直すべきか

自動修正の前に、まず「どこを見るべきか」を推定する必要がある。

- どのQAホストで起きているか
- qa / staging / prod のどれか
- サーバー起因か、クライアント起因か
- どのプロダクト領域か
- どのrepoを見るべきか
- どのログを見るべきか
- 仕様なのか、バグなのか

自動修正の難しさは、パッチを書くことよりも、どこを直すべきかに辿り着くことだった。

難しかったこと2: 情報はJiraだけじゃない

Jiraチケットは入口として優秀だが、修正に必要な情報は分散している。

- Jira: 現象、再現手順、優先度、コメント
- Grafana / Loki: 実際のエラー、頻度、発生時刻
- GitHub: 実装、過去PR、関連Issue
- Slack: 補足説明、現場の会話、暫定対応
- Docs: 仕様、運用ルール

AIにコードを書かせる前に、AIが判断できる文脈を揃える必要がある。

難しかったこと3: 実装できることと、運用できることは違う

コードはほぼできたが、運用に載せる段階で別の壁が見えてきた。

- Jiraには秘匿情報が多い
- Jira API / MCP利用には許可が必要
- Jiraの閲覧権限とGitHubの修正権限が一致しない
- Grafana logs参照権限も慎重に扱う必要がある
- GitHub App / PAT / PR作成権限をどう扱うか
- AIがどこまで読めるか、どこまで書けるか

権限のミスマッチ

たとえば、JiraとGitHubでは権限の考え方が違う。

- Jiraチケットは見えるが、対象repoには書けない
- GitHub Issueは作れるが、PR作成までは許可したくない
- Grafana logsは見たいが、秘匿情報をAIに渡してよいかは別問題
- 個人権限で動かすのか、bot権限で動かすのか

AIエージェントに権限を渡すことは、便利さとリスクを同時に渡すことだった。

SRE・IT戦略との調整

今回の取り組みでは、実装以外にも調整が必要になった。

- Jira / Atlassian MCP を使うための利用許可
- GitHub App / PAT / Copilot assign / Issue作成権限
- Grafana logs 参照権限
- Cloud Run上での実行権限
- Secret管理
- 監査ログやトレーサビリティ

AIエージェントの設計は、コードだけでは完結しない。

ここまでで見えてきたこと

AI活用というと、モデルやプロンプトに目が行きがち。

でも、業務に載せるAIエージェントでは、以下が同じくらい重要だった。

- 何を起点に動くか
- どの情報を取りに行くか
- どこまで自動化してよいか
- 失敗時にどう追跡するか
- 誰が最終判断するか
- 社内の権限設計とどう整合させるか

いきなり完全自動化しない

最初からPR作成・修正完了まで全自動にすると危険。

段階的に信頼できる範囲を増やす。

1. チケットを検知する
2. 調査対象として選定する
3. 情報を集めて要約する
4. GitHub Issueを作成する
5. 修正方針を出す
6. PRを作る
7. 低リスクなものだけ自動化する

今は、Issue作成・調査支援を中心に運用へ載せる段階。

Slackについての考え方

Slack連携はやりたいが、メイントピックではない。

今考えているのは、Slackを「全件通知先」ではなく、人間との接点として使うこと。

- 全チケットを流すとnoisy
- Qualixが選定したものだけ通知したい
- 調査結果や承認を集約する場所にできそう
- ただし、まずはJira起点のフローを安定させるのが優先

横展開・OSS化を見据えた課題

backendだけでなく、車載、決済、支払い請求基盤、IoT基盤などにも広げたい。

ただし、各チームで差分が大きい。

- チケットの書き方
- repo構成
- ログの見方
- QA環境の作り方
- 通知先
- 承認フロー

コアは薄く、現場知識はYAML設定に寄せたい。

発表としての現在地

Qualixは、完成した魔法の自動修正botではない。

今は、以下が見えてきた段階。

- コードとしてはかなり形になった
- Jira起点でAI Agentを動かす構成は作れた
- ただし、実運用には権限・監査・通知・承認が必要
- AIに任せる範囲を段階的に広げる必要がある

試行錯誤も含めて、AIエージェントを現場に入れるリアルな話として共有したい。

学び1: AIに任せるほど、前処理が重要

AIが賢いかどうかだけではなく、AIが判断できる材料を揃えられているかが重要だった。

- Jira本文だけでは足りない
- service catalog が必要
- logs / repo / 過去Issue の文脈が必要
- AIが迷った時に止まれる設計が必要

AIに作らせるからこそ、人間側の設計が重要になる。

学び2: 社内調整もアーキテクチャ

AIエージェントのアーキテクチャは、コード・モデル・MCPだけではない。

- 権限
- 監査
- 秘匿情報
- 通知
- 承認
- 責任分界

これらを含めて、はじめて業務に載る。

学び3: まずは人間が判断しやすい状態を作る

完全自動化を急ぐより、まずは人間が動きやすい状態を作るのが価値になりそう。

- チケットを検知する
- 関連情報を集める
- repo候補を出す
- 原因仮説をまとめる
- GitHub Issueに落とす

これだけでも、初動の認知負荷をかなり下げられる。

今後やりたいこと

- 実運用に向けたSRE・IT戦略との調整
- 権限設計と監査ログの整理
- Grafana / GitHub / Jira MCPの実環境接続安定化
- 品管チケットの対象判定精度向上
- GitHub Issueの品質向上
- 将来的なPR作成
- 他チームへの横展開

まとめ

コード生成AIにより、実装速度はかなり上がった。

しかし、業務で価値を出すには、コードを書く前の仕事をAIに渡す必要がある。

Qualixは、Jiraの品管チケットを起点に、調査・原因推定・GitHub Issue / PR作成へつなげる試み。

難しかったのは、モデルよりも、文脈収集・権限・通知・承認・運用導線だった。

最後に

AIにコードを書かせること自体は、どんどん簡単になっている。

ただし、実際の業務で価値を出すには、チケット検知、文脈収集、repo推定、ログ確認、権限、通知、承認まで含めて設計する必要がある。

Qualixは、Jiraの品管チケットを起点に、その一連の流れをAIに渡していくための試行錯誤です。